Non-Dominated Points (Maxima of a Point Set)

Progress Report

Arun Dinesh Jana
Matrikel-Nr.: 2227229
Bergische Universität Wuppertal

November 5, 2026

Abstract

This report documents the progress made so far on the implementation and experimental study of the *maxima-of-a-point-set* problem (also known as computing the *non-dominated* or *Pareto-optimal* points of a set). A C++17 divide-and-conquer implementation of the Maxima2 algorithm and its Filter/Filter2 merge subroutine has been completed, following the classical treatment of the maxima problem by Preparata and Shamos. The implementation has been validated against hand-constructed test datasets and benchmarked empirically across varying input sizes. This document summarizes the problem definition, algorithmic approach, implementation structure, validation methodology, and the benchmarking results obtained to date, and outlines the parallelization work (MPI, CUDA) planned as the next stage of the project.

Contents

1 Problem Statement **2**

2 Algorithmic Approach **2**
2.1 Dominance Relation 2
2.2 Maxima2: Divide-and-Conquer Maxima 3
2.3 Filter and Filter2: The Merge Step 5

3 Implementation **10**
3.1 Data Structures 11
3.2 Repository Structure 11
3.3 Random Data Generation 12
3.4 Build System 13

4 Validation **19**

5 Benchmarking **25**
5.1 Methodology 25
5.2 Results So Far 28
5.3 Benchmarking Across Dimensions 28

6 Progress Summary **29**

7 Future Scope **29**

1 Problem Statement

Given a set S of N points in E^d (i.e., d-dimensional Euclidean space with integer coordinates), a point p *dominates* a point q if p is greater than or equal to q in every coordinate, with strict inequality in at least one coordinate:

$$p \succ q \iff \big(\forall i \in \{1, \ldots, d\} : p_i \geq q_i\big) \ \wedge \ \big(\exists i \in \{1, \ldots, d\} : p_i > q_i\big). \tag{1}$$

A point is a *maximum* (equivalently, *non-dominated* or *Pareto-optimal*) point of S if no other point in S dominates it. The *maxima problem* asks for the set of all such points. This is a classical problem in computational geometry with applications in multi-objective optimization, database skyline queries, and decision theory.

2 Algorithmic Approach

The project implements the divide-and-conquer Maxima2 algorithm together with its recursive Filter merge step, as described in Preparata and Shamos, *Computational Geometry: An Introduction*.

2.1 Dominance Relation

The dominance predicate defined above is implemented directly as a coordinate-wise comparison over the first d coordinates of two points.

Listing 1: Dominance predicate

```cpp
#include "dominance.hpp"

bool Dominates(const Item& a, const Item& b, size_t d)
{
    bool strict = false;

    for (size_t i=0; i < d; i++)
    {
        if(a.coords[i] < b.coords[i])
        {
            return false;
        }

        if(a.coords[i] > b.coords[i])
        {
            strict = true;
        }
    }

    return strict;
}

bool WeaklyDominates(const Item& a, const Item& b, size_t d)
{
    for (size_t i = 0; i < d; i++)
    {
        if(a.coords[i] < b.coords[i])
        {
            return false;
        }
    }
```

```
32
33      return true;
34  }
```

2.2 Maxima2: Divide-and-Conquer Maxima

The algorithm proceeds as follows:

1. Sort the point set by its last coordinate.
2. Split the sorted set at the median, adjusting the split point so that all points tied on the last coordinate fall on the same side (this preserves correctness of the recursive step).
3. Recursively compute the maxima of the "lower" half S_1 and the "upper" half S_2, obtaining M_1 and M_2 respectively.
4. **Merge**: every point of M_2 is automatically a maximum of S. A point of M_1 survives only if it is not dominated, on the remaining $d-1$ coordinates, by any point of M_2 — this check is delegated to Filter.
5. If the recursion bottoms out with all points tied on the last coordinate (i.e., the median split fails to separate the set), the maxima are computed directly by brute-force pairwise comparison.

Listing 2: Maxima2 divide-and-conquer driver

```
1   #include "maxima2.hpp"
2
3   #include <stdexcept>
4   #include <utility>
5   #include <vector>
6
7   #include "brute_force_maxima.hpp"
8   #include "filter.hpp"
9   #include "point_set.hpp"
10  #include "sorting.hpp"
11
12  namespace {
13
14  // Recursive divide-and-conquer step. The maxima of the range are compacted
        to
15  // its front and their number is returned, so no level of the recursion has
        to
16  // allocate a subset: every operation partitions or compacts in place.
17  //
18  // Precondition: the range is sorted by the last coordinate and every point
        has
19  // d coordinates.
20  size_t MaximaOfSorted(Item* first, size_t count, size_t d)
21  {
22      if (count <= 1) // Base case: if there's 0 or 1 point, return it as the
            maxima
23      {
24          return count;
25      }
26
27      size_t mid = count / 2; // Finds the midpoint of the sorted points
28
```

```
29      // Move mid to the right until we find a point with a different last
            coordinate
30      // This ensures that we don't split points with the same last
            coordinate
31
32      while (mid < count && first[mid].coords[d - 1] == first[mid - 1].coords
            [d - 1])
33      {
34          ++mid;
35      }
36
37      // If mid reaches the end of the range,
38      // it means all points have the same last coordinate
39      //
40
41      if (mid == count)
42      {
43          return BruteForceMaximaInPlace(first, count, d);
44      }
45
46      const size_t m1 = MaximaOfSorted(first, mid, d);
47      const size_t m2 = MaximaOfSorted(first + mid, count - mid, d);
48
49      if (d == 1)
50      {
51          // Every maximum is in the upper half, which was sorted to dominate
                .
52          for (size_t i = 0; i < m2; ++i)
53          {
54              first[i] = std::move(first[mid + i]);
55          }
56
57          return m2;
58      }
59
60      // M1 is at [first, first + m1), M2 at [first + mid, first + mid + m2).
61      const size_t survivors = FILTER(first, m1, first + mid, m2, d - 1);
62
63      // Append M2 after the survivors of M1; M2 remains a complete set
            because
64      // FILTER only reorders it.
65      for (size_t i = 0; i < m2; ++i)
66      {
67          if (survivors + i != mid + i)
68          {
69              first[survivors + i] = std::move(first[mid + i]);
70          }
71      }
72
73      return survivors + m2;
74  }
75
76  } // namespace
77
78  std::vector<Item> MAXIMA2(const PointSet& set)
79  {
80      if (!IsValid(set))
81      {
```

```
        throw std::invalid_argument(
            "MAXIMA2: point set has zero dimension or a point with the
                wrong coordinate count");
    }

    std::vector<Item> points = set.points;

    SortByLastCoordinate(points); // Sort points by the last coordinate

    const size_t count = MaximaOfSorted(points.data(), points.size(), set.
        dimensions);

    // The maxima are compacted at the front; dropping the rest avoids
        copying
    // them into a second result vector.
    points.resize(count);

    return points;
}
```

2.3 Filter and Filter2: The Merge Step

Filter(U, V, d) computes the subset of U that is not dominated by any point of V, considering only the first d coordinates. Because the coordinate that has been dropped from the comparison already dominates strictly, a point of V that merely ties on the remaining coordinates still dominates: Filter uses weak dominance there. The general recursive case works by:

- Equipartitioning V on coordinate d into V_1 (lower half) and V_2 (upper half), via PartitionV.
- Deriving a splitting threshold from the maximum d-th coordinate in V_1, via ThresholdFromV1.
- Splitting U at that threshold into U_1 and U_2, via SplitU.
- Recursing on three sub-problems — (U_2, V_2, d), (U_1, V_1, d), and $(U_1, V_2, d-1)$ — and recombining the results. The third sub-problem is applied to the survivors of the second, which computes the same intersection without materialising two intermediate sets.

The recursion bottoms out at $d = 1$ (a simple coordinate threshold test) and at $d = 2$, where Filter2 performs a linear-time plane sweep over the remaining two coordinates.

Listing 3: Filter and Filter2

```
#include "filter.hpp"

#include <algorithm>
#include <limits>
#include <utility>

#include "dominance.hpp"
#include "partition.hpp"
#include "sorting.hpp"
#include "split_u.hpp"
#include "threshold.hpp"

namespace {

```

```
// Keeps the points of U that are not weakly dominated by a single point,
// compacting them to the front.
size_t RemoveDominatedByOne(Item* u, size_t u_count, const Item& v, size_t
    d)
{
    size_t write = 0;

    for (size_t i = 0; i < u_count; ++i)
    {
        if (WeaklyDominates(v, u[i], d))
        {
            continue;
        }

        if (write != i)
        {
            u[write] = std::move(u[i]);
        }

        ++write;
    }

    return write;
}

// Single coordinate: a point survives only if it is strictly better than
    every
// point of V, because the coordinate that is no longer compared already
// dominates strictly.
size_t RemoveDominatedByMaxCoordinate(
    Item* u,
    size_t u_count,
    const Item* v,
    size_t v_count)
{
    double maximum = v[0].coords[0];

    for (size_t i = 1; i < v_count; ++i)
    {
        maximum = std::max(maximum, v[i].coords[0]);
    }

    size_t write = 0;

    for (size_t i = 0; i < u_count; ++i)
    {
        if (u[i].coords[0] <= maximum)
        {
            continue;
        }

        if (write != i)
        {
            u[write] = std::move(u[i]);
        }

        ++write;
    }
```

```
71
72      return write;
73  }
74
75  // Linear sweep for the two remaining coordinates. Both ranges are sorted
        by the
76  // second coordinate; walking them downwards keeps the best first
        coordinate
77  // seen so far, so every point needs only two comparisons.
78  size_t Filter2InPlace(Item* u, size_t u_count, Item* v, size_t v_count)
79  {
80      SortByCoordinate(u, u_count, 1);
81      SortByCoordinate(v, v_count, 1);
82
83      double best_x_strict = std::numeric_limits<double>::lowest();
84
85      size_t j = v_count;   // points of V above the current coordinate
86      size_t i = u_count;
87      size_t write = u_count; // survivors are written from the back
88
89      while (i > 0)
90      {
91          const double y = u[i - 1].coords[1];
92
93          // Absorb every v with strictly greater y into best_x_strict.
94          while (j > 0 && v[j - 1].coords[1] > y)
95          {
96              best_x_strict = std::max(best_x_strict, v[j - 1].coords[0]);
97              --j;
98          }
99
100         // Points of V tied at this y can still dominate through a larger x
                .
101         size_t k = j;
102         double best_x_equal = std::numeric_limits<double>::lowest();
103
104         while (k > 0 && v[k - 1].coords[1] == y)
105         {
106             best_x_equal = std::max(best_x_equal, v[k - 1].coords[0]);
107             --k;
108         }
109
110         while (i > 0 && u[i - 1].coords[1] == y)
111         {
112             const bool dominated =
113                 best_x_strict >= u[i - 1].coords[0] ||
114                 best_x_equal >= u[i - 1].coords[0];
115
116             if (!dominated)
117             {
118                 --write;
119
120                 if (write != i - 1)
121                 {
122                     u[write] = std::move(u[i - 1]);
123                 }
124             }
125
```

```
            --i;
        }

        best_x_strict = std::max(best_x_strict, best_x_equal);

        j = k;
    }

    const size_t survivors = u_count - write;

    // The survivors form a contiguous block at the back; move it to the
       front.
    for (size_t t = 0; t < survivors; ++t)
    {
        if (write + t != t)
        {
            u[t] = std::move(u[write + t]);
        }
    }

    return survivors;
}

} // namespace

size_t FILTER(Item* u, size_t u_count, Item* v, size_t v_count, size_t d)
{
    if (u_count == 0)
    {
        return 0;
    }

    if (v_count == 0)
    {
        return u_count;
    }

    if (v_count == 1)
    {
        return RemoveDominatedByOne(u, u_count, v[0], d);
    }

    if (d == 1)
    {
        return RemoveDominatedByMaxCoordinate(u, u_count, v, v_count);
    }

    if (d == 2)
    {
        return Filter2InPlace(u, u_count, v, v_count);
    }

    const size_t v1_count = PartitionV(v, v_count, d);
    const size_t v2_count = v_count - v1_count;

    const double threshold = ThresholdFromV1(v, v1_count, d);

    const size_t u1_count = SplitU(u, u_count, threshold, d);
```

```
    const size_t u2_count = u_count - u1_count;

    Item* const v2 = v + v1_count;

    // U2 against V2, and U1 against V1, on the full set of coordinates.
    const size_t a_count = FILTER(u + u1_count, u2_count, v2, v2_count, d);
    const size_t b_count = FILTER(u, u1_count, v, v1_count, d);

    // U1 against V2 with one fewer coordinate. Filtering the survivors of
       the
    // previous step is the same predicate as intersecting the two survivor
    // sets, without materialising either of them.
    const size_t c_count = FILTER(u, b_count, v2, v2_count, d - 1);

    for (size_t i = 0; i < a_count; ++i)
    {
        if (c_count + i != u1_count + i)
        {
            u[c_count + i] = std::move(u[u1_count + i]);
        }
    }

    return c_count + a_count;
}
```

Listing 4: Equipartition of V

```
#include "partition.hpp"
#include "sorting.hpp"

size_t PartitionV(Item* points, size_t count, size_t d)
{
    SortByCoordinate(points, count, d - 1);

    return count / 2;
}
```

Listing 5: Threshold extraction from V_1

```
#include "threshold.hpp"

double ThresholdFromV1(const Item* points, size_t count, size_t d)
{
    double maximum = points[0].coords[d - 1];

    for (size_t i = 1; i < count; ++i)
    {
        if (points[i].coords[d - 1] > maximum)
        {
            maximum = points[i].coords[d - 1];
        }
    }

    return maximum;
}
```

Listing 6: Threshold-based split of U

```
#include "split_u.hpp"
```

```

#include <algorithm>

size_t SplitU(Item* points, size_t count, double threshold, size_t d)
{
    Item* const split = std::partition(
        points,
        points + count,
        [threshold, d](const Item& point)
        {
            return point.coords[d - 1] <= threshold;
        }
    );

    return static_cast<size_t>(split - points);
}
```

Listing 7: Coordinate-based sorting helpers

```
#include "sorting.hpp"

#include <algorithm>

void SortByLastCoordinate(Item* first, size_t count)
{
    std::sort(first, first + count, [](const Item& a, const Item& b)
        {
            return a.coords.back() < b.coords.back();
        }
    );
}

void SortByCoordinate(Item* first, size_t count, size_t k)
{
    std::sort(first, first + count, [k](const Item& a, const Item& b)
        {
            return a.coords[k] < b.coords[k];
        }
    );
}

void SortByLastCoordinate(std::vector<Item>& items)
{
    SortByLastCoordinate(items.data(), items.size());
}

void SortByCoordinate(std::vector<Item>& items, size_t k)
{
    SortByCoordinate(items.data(), items.size(), k);
}
```

3 Implementation

The implementation is written in C++17 and organized as a small modular library plus the benchmark and test programs built with CMake. Both MAXIMA2 and FILTER operate in place on disjoint ranges of one contiguous buffer: each recursive step partitions its range and compacts its result to the front, so no level of the recursion copies its input.

3.1 Data Structures

Points are represented by a simple aggregate:

Listing 8: Point representation

```cpp
#pragma once

#include <vector>

// A point in d-dimensional space.
//
// Coordinates are compared exactly (no tolerance), so inputs are expected to
// carry the values that should be compared; the algorithms are not designed for
// values that are only approximately equal.
struct Item {
    std::vector<double> coords; // coords[0]..coords[d-1]
};
```

3.2 Repository Structure

```
non-dominated-points/
|-- src/
|   |-- main.cpp                (MAXIMA2 benchmark driver)
|   |-- main_bruteforce.cpp     (brute-force benchmark driver)
|   |-- benchmark_runner.cpp    (shared loading and timing)
|   |-- benchmark.cpp           (CSV output)
|   |-- point_set.cpp           (dimension validation)
|   |-- dominance.cpp
|   |-- sorting.cpp
|   |-- maxima2.cpp
|   |-- brute_force_maxima.cpp
|   |-- filter.cpp
|   |-- partition.cpp
|   |-- split_u.cpp
|   |-- threshold.cpp
|   `-- print.cpp
|-- include/                    (corresponding headers)
|-- mpi_src/                    (ring, fan-in/fan-out, marry-go-round)
|-- DataSets/                   (benchmark inputs, 5D-10D)
|-- scripts/                    (dataset generator, sweep and job scripts)
|-- validation/
|   |-- validator.cpp
|   |-- validation_main.cpp
|   |-- differential_test.cpp
|   `-- test_datasets/          (Test1.txt .. Test16.txt)
|-- plots/
|   |-- plot_results.py
|   |-- plot_mpi_results.py
|   `-- figures/
|-- results/                    (generated benchmark CSVs)
`-- CMakeLists.txt
```

3.3 Random Data Generation

Benchmark inputs are generated by drawing n points with D coordinates, each independently uniform on $[0, 1]$ and rounded to six decimals for storage, by a short Python script.

Listing 9: Random point generation

```python
"""Generates the benchmark datasets in DataSets/.

    python3 scripts/create_testcases.py             # every dimension and size
    python3 scripts/create_testcases.py --seed 7    # reproducible run

Each dataset holds `num_points` points with `dimensions` coordinates, drawn uniformly from [0, 1] and rounded to six decimals for storage.
"""

import argparse
import os
import random
import time

DIMENSIONS = [5, 6, 7, 8, 9, 10]
POINT_COUNTS = [50, 100, 500, 1000, 10000, 100000, 1000000]

REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))


def generate_test_case(dimensions, num_points, filename=None):
    lines = []

    # Header
    lines.append(f"{dimensions} {num_points}")

    # Generate points
    for _ in range(num_points):
        point = [
            f"{random.random():.6f}"
            for _ in range(dimensions)
        ]
        lines.append(" ".join(point))

    # save to file
    if filename:
        os.makedirs(os.path.dirname(filename), exist_ok=True)
        with open(filename, "w") as f:
            f.write("\n".join(lines) + "\n")


def main():
    parser = argparse.ArgumentParser(description="Generate the benchmark datasets.")
    parser.add_argument(
        "--out",
        default=os.path.join(REPO_ROOT, "DataSets"),
        help="directory to write the <D>D_Data/ folders into",
    )
    parser.add_argument(
        "--dimensions",
```

```
        type=int,
        nargs="+",
        default=DIMENSIONS,
        help="dimensions to generate",
    )
    parser.add_argument(
        "--points",
        type=int,
        nargs="+",
        default=POINT_COUNTS,
        help="point counts to generate, each written as its matching Test<N
            >",
    )
    parser.add_argument(
        "--seed",
        type=int,
        default=None,
        help="seed for a reproducible run; defaults to the current time",
    )
    args = parser.parse_args()

    unknown = sorted(set(args.points) - set(POINT_COUNTS))
    if unknown:
        parser.error(
            f"point counts {unknown} have no Test<N> name; "
            f"expected a subset of {POINT_COUNTS}"
        )

    random.seed(args.seed if args.seed is not None else int(time.time()))

    for dimensions in args.dimensions:
        for test, num_points in enumerate(POINT_COUNTS, start=1):
            if num_points not in args.points:
                continue
            filename = os.path.join(
                args.out, f"{dimensions}D_Data", f"Test{test}.txt"
            )
            generate_test_case(
                dimensions=dimensions,
                num_points=num_points,
                filename=filename,
            )
            print(f"{dimensions}D {num_points:>8} points -> {filename}")


if __name__ == "__main__":
    main()
```

3.4 Build System

The project is built with CMake and produces the MAXIMA2 and brute-force benchmark drivers (`Maxima_method`, `brute_force`), the `validator` and `differential_test` test programs, and — when an MPI installation is available — the three MPI programs (`MPI_Ringmethod`, `MPI_FanInOut`, `MPI_Marrygomethod`).

Listing 10: CMakeLists.txt

```
cmake_minimum_required(VERSION 3.15)
```

```
2
3  project(NonDominatedPoints
4      VERSION 0.1.0
5      DESCRIPTION "Maxima (non-dominated points) of a d-dimensional point set
           "
6      LANGUAGES CXX
7  )
8
9  #
       ------------------------------------------------------------------------
10 # Options
11 #
       ------------------------------------------------------------------------
12 option(NDP_ENABLE_MPI    "Build the MPI programs (requires an MPI
       installation)" ON)
13 option(NDP_BUILD_TESTS   "Build the validator and register it with CTest"
           ON)
14 option(NDP_SANITIZE      "Enable AddressSanitizer +
       UndefinedBehaviorSanitizer"  OFF)
15 option(NDP_WARNINGS_AS_ERRORS "Treat compiler warnings as errors"
                  OFF)
16
17 #
       ------------------------------------------------------------------------
18 # Language standard and default build type
19 #
       ------------------------------------------------------------------------
20 set(CMAKE_CXX_STANDARD 17)
21 set(CMAKE_CXX_STANDARD_REQUIRED ON)
22 set(CMAKE_CXX_EXTENSIONS OFF)
23
24 get_property(_is_multi_config GLOBAL PROPERTY GENERATOR_IS_MULTI_CONFIG)
25 if(NOT _is_multi_config AND NOT CMAKE_BUILD_TYPE)
26     set(CMAKE_BUILD_TYPE Release CACHE STRING "Build type" FORCE)
27     set_property(CACHE CMAKE_BUILD_TYPE PROPERTY STRINGS
28         Debug Release RelWithDebInfo MinSizeRel)
29 endif()
30
31 #
       ------------------------------------------------------------------------
32 # Shared compile options (warnings / sanitizers) as an interface target.
33 # Optimisation level comes from CMAKE_BUILD_TYPE (Release => -O3 -DNDEBUG).
34 #
       ------------------------------------------------------------------------
35 add_library(ndp_options INTERFACE)
36
37 if(CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang|AppleClang")
38     target_compile_options(ndp_options INTERFACE
39         -Wall -Wextra -Wpedantic -Wshadow -Wconversion
40     )
41     if(NDP_WARNINGS_AS_ERRORS)
42         target_compile_options(ndp_options INTERFACE -Werror)
```

```
    endif()
    if(NDP_SANITIZE)
        target_compile_options(ndp_options INTERFACE
            -fsanitize=address,undefined -fno-omit-frame-pointer)
        target_link_options(ndp_options INTERFACE
            -fsanitize=address,undefined)
    endif()
elseif(MSVC)
    target_compile_options(ndp_options INTERFACE /W4)
    if(NDP_WARNINGS_AS_ERRORS)
        target_compile_options(ndp_options INTERFACE /WX)
    endif()
endif()

#
    ------------------------------------------------------------------------

# Core library
#
    ------------------------------------------------------------------------

add_library(nondom STATIC
    src/antichain.cpp
    src/benchmark.cpp
    src/benchmark_runner.cpp
    src/brute_force_maxima.cpp
    src/dataset.cpp
    src/dominance.cpp
    src/filter.cpp
    src/maxima2.cpp
    src/partition.cpp
    src/point_set.cpp
    src/print.cpp
    src/sorting.cpp
    src/split_u.cpp
    src/threshold.cpp
)
target_include_directories(nondom PUBLIC
    $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
)
target_link_libraries(nondom PUBLIC ndp_options)

#
    ------------------------------------------------------------------------

# Serial executables
#
    ------------------------------------------------------------------------

add_executable(Maxima_method src/main.cpp)
target_link_libraries(Maxima_method PRIVATE nondom)

add_executable(brute_force src/main_bruteforce.cpp)
target_link_libraries(brute_force PRIVATE nondom)

# Peak-memory and million-point measurements; the quadratic baseline cannot
    run
# these datasets, so they are measured on their own.
```

```
add_executable(large_benchmark tools/large_benchmark.cpp)
target_link_libraries(large_benchmark PRIVATE nondom)

# Worst case for both algorithms: an antichain, where every point is maximal
# and nothing can be pruned early.
add_executable(worst_case_benchmark tools/worst_case_benchmark.cpp)
target_link_libraries(worst_case_benchmark PRIVATE nondom)

# The same antichain written out as a dataset file, so the MPI programs can be
# measured on the identical input.
add_executable(antichain_dataset tools/antichain_dataset.cpp)
target_link_libraries(antichain_dataset PRIVATE nondom)

#
    ----------------------------------------------------------------------------

# Validation / tests
#
    ----------------------------------------------------------------------------

if(NDP_BUILD_TESTS)
    enable_testing()

    # Every test gets a time bound. A hung MPI runtime (or a deadlocked
    # algorithm) must show up as one failed test within seconds, not as a suite
    # that never returns.
    function(ndp_add_test name timeout_seconds)
        add_test(NAME ${name} COMMAND ${ARGN})
        set_tests_properties(${name} PROPERTIES TIMEOUT ${timeout_seconds})
    endfunction()

    add_executable(validator
        validation/validation_main.cpp
        validation/validator.cpp
    )
    target_link_libraries(validator PRIVATE nondom)

    # The validator loads "../validation/test_datasets/TestN.txt", so run it
    # from <source>/validation. This works for any build directory location.
    ndp_add_test(validator 120 validator)
    set_tests_properties(validator PROPERTIES
        WORKING_DIRECTORY ${CMAKE_CURRENT_SOURCE_DIR}/validation
    )
    # validator exits non-zero on failure; also check its case count so that a
    # dataset that silently fails to load cannot reduce the coverage unnoticed.
    set_tests_properties(validator PROPERTIES
        PASS_REGULAR_EXPRESSION "Passed : 16 / 16"
    )

    # Randomized differential test against the brute-force reference. Runs in
```

```
    # memory, so it needs no data files and no particular working directory.
    add_executable(differential_test validation/differential_test.cpp)
    target_link_libraries(differential_test PRIVATE nondom)

    ndp_add_test(differential_test 60 differential_test)

    # The dominance predicate, on its own. Both algorithms share it, so a
    # difference between them cannot expose a mistake in it.
    add_executable(dominance_test validation/dominance_test.cpp)
    target_link_libraries(dominance_test PRIVATE nondom)
    ndp_add_test(dominance_relation 60 dominance_test)

    # Inputs the random test never generates: empty, single point, duplicates,
    # one dimension, everything in one tie group.
    add_executable(edge_case_test validation/edge_case_test.cpp)
    target_link_libraries(edge_case_test PRIVATE nondom)
    ndp_add_test(edge_cases 60 edge_case_test)

    # The merge helpers (PartitionV / SplitU / FILTER) with their contracts
    # spelled out, including the weak-dominance tie rule FILTER depends on.
    add_executable(partition_filter_test validation/partition_filter_test.cpp)
    target_link_libraries(partition_filter_test PRIVATE nondom)
    ndp_add_test(partition_filter 60 partition_filter_test)

    # Pipeline tests: the loader's refusal paths, the CSV upsert behaviour the
    # plots depend on, the suite's handling of missing datasets, and the
    # antichain generator behind the worst-case measurements. All of them write
    # to a temporary directory only.
    add_executable(dataset_test validation/dataset_test.cpp)
    target_link_libraries(dataset_test PRIVATE nondom)
    ndp_add_test(dataset_loader 60 dataset_test)

    add_executable(benchmark_csv_test validation/benchmark_csv_test.cpp)
    target_link_libraries(benchmark_csv_test PRIVATE nondom)
    ndp_add_test(benchmark_pipeline 60 benchmark_csv_test)

    add_executable(antichain_test validation/antichain_test.cpp)
    target_link_libraries(antichain_test PRIVATE nondom)
    ndp_add_test(antichain_generator 60 antichain_test)
endif()

#
    ------------------------------------------------------------------------

# MPI executables (optional)
#
    ------------------------------------------------------------------------

if(NDP_ENABLE_MPI)
    find_package(MPI COMPONENTS CXX)

    if(MPI_CXX_FOUND)
        # Standalone: flat-array implementation, does not use the core
```

```
    library.
187 add_executable(MPI_Marrygomethod mpi_src/mpi_marry_go_round_method.
    cpp)
188 target_link_libraries(MPI_Marrygomethod PRIVATE MPI::MPI_CXX
    ndp_options)
189
190 # These use Item / Dominates / BruteForceMaxima from the core
    library.
191 add_executable(MPI_Ringmethod mpi_src/mpi_ring_method.cpp)
192 target_link_libraries(MPI_Ringmethod PRIVATE MPI::MPI_CXX nondom)
193
194 add_executable(MPI_FanInOut mpi_src/mpi_fan_in_out.cpp)
195 target_link_libraries(MPI_FanInOut PRIVATE MPI::MPI_CXX nondom)
196
197 # Correctness of the distributed programs. They only ever printed
    their
198 # result, so nothing checked them automatically; each is now run on
     an
199 # 8-point fixture whose answer is 4 maxima, at the process counts
    where
200 # the token passing wraps around. The marry-go-round program has a
201 # separate code path at one process, which is covered as well.
202 if(NDP_BUILD_TESTS AND MPIEXEC_EXECUTABLE)
203     set(MPI_TEST_DATASET
204         ${CMAKE_CURRENT_SOURCE_DIR}/validation/test_datasets/
            MPI_Test.txt)
205
206     # The marry-go-round program writes its CSV relative to the
        parent
207     # of the working directory.
208     file(MAKE_DIRECTORY ${CMAKE_CURRENT_BINARY_DIR}/mpi_work)
209     file(MAKE_DIRECTORY ${CMAKE_CURRENT_BINARY_DIR}/results)
210
211     # The bound is generous for eight points, but it is what turns
        a
212     # deadlock (or an MPI runtime that hangs in MPI_Finalize) into
        a
213     # failed test instead of a suite that has to be interrupted.
214     foreach(count 1 2 3)
215         ndp_add_test(mpi_marry_go_round_p${count} 20
216             ${MPIEXEC_EXECUTABLE} ${MPIEXEC_NUMPROC_FLAG}
217             ${count} ${MPIEXEC_PREFLAGS}
218             $<TARGET_FILE:MPI_Marrygomethod>
219             ${MPI_TEST_DATASET})
220         set_tests_properties(mpi_marry_go_round_p${count}
            PROPERTIES
221             WORKING_DIRECTORY ${CMAKE_CURRENT_BINARY_DIR}/mpi_work
222             PASS_REGULAR_EXPRESSION "maxima=4")
223     endforeach()
224
225     foreach(program MPI_Ringmethod MPI_FanInOut)
226         foreach(count 2 3)
227             ndp_add_test(${program}_p${count} 20
228                 ${MPIEXEC_EXECUTABLE} ${MPIEXEC_NUMPROC_FLAG}
229                 ${count} ${MPIEXEC_PREFLAGS}
230                 $<TARGET_FILE:${program}>
231                 ${MPI_TEST_DATASET})
232             set_tests_properties(${program}_p${count} PROPERTIES
```

```
                        WORKING_DIRECTORY ${CMAKE_CURRENT_BINARY_DIR}/
                            mpi_work
                        PASS_REGULAR_EXPRESSION "Final maxima: 4")
                endforeach()
            endforeach()
        elseif(NDP_BUILD_TESTS)
            message(WARNING
                "mpiexec not found: the MPI correctness tests are not
                    registered.")
        endif()
    else()
        message(WARNING
            "MPI not found: skipping MPI programs. "
            "Install MPI or configure with -DNDP_ENABLE_MPI=OFF.")
    endif()
endif()
```

4 Validation

Correctness of MAXIMA2 is checked against 16 hand-constructed test datasets stored in `validation/test_datas`
Each dataset file specifies the dimension, the input point set, and the expected set of maxima,
e.g.:

```
# Dimension
2

# Number of Input Points
1

# Input Points
5 5

# Number of Expected Maxima
1

# Expected Maxima
5 5
```

The dataset loader parses this format and returns the input points alongside the expected output for comparison.

Listing 11: Test dataset loader

```
#include "validator.hpp"

#include <fstream>
#include <iostream>
#include <optional>
#include <sstream>
#include <string>

namespace {

// Dataset files separate sections with blank lines and '#' comments.
    Returns
```

```cpp
// the next line that carries data.
bool NextDataLine(std::istream& file, std::string& line)
{
    while (std::getline(file, line))
    {
        if (line.find_first_not_of(" \t\r") == std::string::npos)
        {
            continue;
        }

        if (line[0] == '#')
        {
            continue;
        }

        return true;
    }

    return false;
}

std::optional<size_t> ReadCount(std::istream& file)
{
    std::string line;

    if (!NextDataLine(file, line))
    {
        return std::nullopt;
    }

    std::istringstream fields(line);
    size_t value = 0;

    if (!(fields >> value))
    {
        return std::nullopt;
    }

    fields >> std::ws;

    if (!fields.eof())
    {
        return std::nullopt;
    }

    return value;
}

// Reads `count` point lines, each carrying exactly `dimension` coordinates
    .
bool ReadPoints(
    std::istream& file,
    size_t dimension,
    size_t count,
    std::vector<Item>& points
)
{
    for (size_t i = 0; i < count; ++i)
```

```
    {
        std::string line;

        if (!NextDataLine(file, line))
        {
            return false;
        }

        std::istringstream fields(line);

        Item item;
        double coordinate = 0.0;

        while (fields >> coordinate)
        {
            item.coords.push_back(coordinate);
        }

        if (!fields.eof() || item.coords.size() != dimension)
        {
            return false;
        }

        points.push_back(item);
    }

    return true;
}

} // namespace

std::optional<ValidationDataset> LoadDataset(const std::string& filename)
{
    std::ifstream file(filename);

    if (!file)
    {
        std::cerr << "Could not open dataset: " << filename << '\n';

        return std::nullopt;
    }

    ValidationDataset dataset;

    const std::optional<size_t> dimension = ReadCount(file);

    if (!dimension || *dimension == 0)
    {
        std::cerr << "Missing or invalid dimension: " << filename << '\n';

        return std::nullopt;
    }

    dataset.dimension = *dimension;

    const std::optional<size_t> input_count = ReadCount(file);

    if (!input_count)
```

```
    {
        std::cerr << "Missing or invalid input point count: " << filename
            << '\n';

        return std::nullopt;
    }

    if (!ReadPoints(file, dataset.dimension, *input_count, dataset.
        inputPoints))
    {
        std::cerr << "Malformed input points: " << filename << '\n';

        return std::nullopt;
    }

    const std::optional<size_t> expected_count = ReadCount(file);

    if (!expected_count)
    {
        std::cerr << "Missing or invalid expected maxima count: " <<
            filename << '\n';

        return std::nullopt;
    }

    if (!ReadPoints(file, dataset.dimension, *expected_count, dataset.
        expectedMaxima))
    {
        std::cerr << "Malformed expected maxima: " << filename << '\n';

        return std::nullopt;
    }

    return dataset;
}
```

Listing 12: Header for validation types

```
#pragma once

#include <optional>
#include <string>
#include <vector>

#include "item.hpp"

struct ValidationDataset
{
    size_t dimension = 0;
    std::vector<Item> inputPoints;
    std::vector<Item> expectedMaxima;
};

// Loads a test dataset (dimension, points, expected maxima). Returns
    nothing
// when the file cannot be opened or does not follow the documented format,
     and
// reports the reason on stderr.
std::optional<ValidationDataset> LoadDataset(const std::string& filename);
```

The validation driver runs MAXIMA2 on all 16 test cases and reports a pass/fail summary, printing the expected and computed maxima side by side whenever a mismatch occurs. Expected maxima are compared as an unordered set, since the order in which maxima are listed carries no meaning.

Listing 13: Validation driver

```cpp
#include <algorithm>
#include <iostream>
#include <optional>
#include <vector>

#include "maxima2.hpp"
#include "point_set.hpp"
#include "print.hpp"
#include "validator.hpp"

namespace {

constexpr int kFirstDataset = 1;
constexpr int kLastDataset = 16;

// The maxima of a point set are an unordered set: compare as multisets rather
// than relying on the order in which either side happens to list them.
bool CompareMaxima(const std::vector<Item>& computed, const std::vector<Item>& expected)
{
    if (computed.size() != expected.size())
    {
        return false;
    }

    auto sorted = [](std::vector<Item> items)
    {
        std::sort(items.begin(), items.end(),
            [](const Item& a, const Item& b)
            {
                return a.coords < b.coords;
            }
        );

        return items;
    };

    const std::vector<Item> a = sorted(computed);
    const std::vector<Item> b = sorted(expected);

    return std::equal(a.begin(), a.end(), b.begin(),
        [](const Item& x, const Item& y)
        {
            return x.coords == y.coords;
        }
    );
}

} // namespace
```

```cpp

int main()
{
    int passed = 0;
    const int total = kLastDataset - kFirstDataset + 1;

    for (int i = kFirstDataset; i <= kLastDataset; i++)
    {
        const std::string filename =
            "../validation/test_datasets/Test" + std::to_string(i) + ".txt"
                ;

        const std::optional<ValidationDataset> dataset = LoadDataset(
            filename);

        if (!dataset)
        {
            std::cout << "Test Case " << i << " : SKIPPED (could not load "
                 << filename << ")\n\n";

            continue;
        }

        const PointSet set{dataset->dimension, dataset->inputPoints};

        const auto maxima = MAXIMA2(set);

        if (CompareMaxima(maxima, dataset->expectedMaxima))
        {
            std::cout << "Test Case " << i << " : PASSED\n\n";
            passed++;
        }
        else
        {
            std::cout << "Test Case " << i << " : FAILED\n";

            std::cout << "\nExpected Maxima\n";
            printItems(dataset->expectedMaxima);

            std::cout << "\nComputed Maxima\n";
            printItems(maxima);

            std::cout << "\n";
        }
    }

    std::cout << "\n====================================\n";
    std::cout << "Validation Summary\n";
    std::cout << "====================================\n";
    std::cout << "Passed : " << passed << " / " << total << "\n";

    return passed == total ? 0 : 1;
}
```

In addition to these hand-built cases, `differential_test` compares MAXIMA2 against the brute-force reference over 13 104 generated cases (dimensions 2–8, point counts 2–40, integer and fractional coordinate ranges chosen to produce ties and duplicate points) with a fixed seed, so that a failure is reproducible. It also checks that inconsistent point sets are rejected instead

of being read out of bounds. Both programs run under CTest.
At the current stage, all 16 test cases pass and the differential test reports no mismatches.

5 Benchmarking

5.1 Methodology

The benchmark drivers load the datasets in `DataSets/` and measure the wall-clock execution time of MAXIMA2 and of the brute-force baseline with `std::chrono::steady_clock`, taking the median of 10 repetitions per configuration. Both drivers cover the same inputs, `Test1` through `Test6` (50 to 100 000 points); each configuration occupies exactly one row, so re-running a dimension replaces its measurements rather than duplicating them. Results are written to `results/maxima_benchmark.csv` and `results/brute_force_benchmark.csv` and subsequently plotted by `plots/plot_results.py` (using pandas and matplotlib) into `plots/figures/`.

Listing 14: Benchmark driver

```cpp
#include <iostream>
#include <string>

#include "benchmark_runner.hpp"

int main(int argc, char* argv[])
{
    if (argc != 2)
    {
        std::cerr << "Usage: " << argv[0] << " <dimension_folder>\n";

        return 1;
    }

    const std::string base_directory = "../DataSets/" + std::string(argv
        [1]);

    return RunBenchmarkSuite(
        base_directory,
        "../results/maxima_benchmark.csv",
        Algorithm::Maxima2
    );
}
```

Listing 15: Plotting script

```python
"""Runtime comparison of brute force, MAXIMA2 and the MPI marry-go-round.

One 2x3 panel figure covers all six dimensions. Every panel carries the
    same
three curves, so the figure replaces the earlier one-file-per-dimension set
without making any single panel busier.
"""

import matplotlib.pyplot as plt

from benchmark_data import (
    BRUTE_FORCE_CSV,
    FIGURES_DIR,
    MARRY_GO_ROUND_CSV,
    MAXIMA_CSV,
```

```
    load_benchmark,
    load_parallel_benchmark,
    require,
)


# Process count drawn for the MPI curve; the benchmark CSV holds every count
# from 1 to 20, and one curve per panel keeps the comparison readable.
MARRY_GO_ROUND_PROCESSES = 8

require(MAXIMA_CSV, BRUTE_FORCE_CSV)

FIGURES_DIR.mkdir(exist_ok=True)

maxima = load_benchmark(MAXIMA_CSV)
brute_force = load_benchmark(BRUTE_FORCE_CSV)

# The MPI results come from a local run and are not tracked, so the third curve
# is optional: the two sequential series still plot without it.
if MARRY_GO_ROUND_CSV.exists():
    marry_go_round = load_parallel_benchmark(
        MARRY_GO_ROUND_CSV, MARRY_GO_ROUND_PROCESSES
    )
else:
    marry_go_round = None
    print(
        f"No MPI results at {MARRY_GO_ROUND_CSV}; "
        "plotting MAXIMA2 and brute force only."
    )

dimensions = sorted(int(d) for d in maxima["dimensions"].unique())

figure, axes = plt.subplots(
    2,
    3,
    figsize=(15, 8),
    squeeze=False,
    sharex=True,
    sharey=True,
)

panels = axes.flatten()

for panel, dim in zip(panels, dimensions):
    maxima_subset = maxima[maxima["dimensions"] == dim].sort_values("num_points")
    brute_force_subset = brute_force[
        brute_force["dimensions"] == dim
    ].sort_values("num_points")

    panel.plot(
        maxima_subset["num_points"],
        maxima_subset["execution_time_s"],
        marker="o",
        linewidth=2,
        color="tab:blue",
```

```
        label="MAXIMA2",
    )

    panel.plot(
        brute_force_subset["num_points"],
        brute_force_subset["execution_time_s"],
        marker="o",
        linewidth=2,
        color="tab:orange",
        label="Brute Force",
    )

    if marry_go_round is not None:
        marry_subset = marry_go_round[
            marry_go_round["dimensions"] == dim
        ].sort_values("num_points")

        if not marry_subset.empty:
            panel.plot(
                marry_subset["num_points"],
                marry_subset["execution_time_s"],
                marker="s",
                linestyle="--",
                linewidth=2,
                color="tab:green",
                label=f"MPI (P={MARRY_GO_ROUND_PROCESSES})",
            )

    panel.set_title(f"{dim}D")
    panel.set_xscale("log")
    panel.set_yscale("log")
    panel.grid(True, which="both", linestyle="--", alpha=0.4)

for panel in panels[len(dimensions):]:
    panel.set_visible(False)

# Axis labels go on the outer panels only; the grid shares its axes.
for panel in panels[-3:]:
    panel.set_xlabel("Number of Points")

for panel in panels[::3]:
    panel.set_ylabel("Execution Time (s)")

handles, labels = panels[0].get_legend_handles_labels()

figure.legend(
    handles,
    labels,
    loc="upper center",
    ncol=len(labels),
    frameon=False,
    bbox_to_anchor=(0.5, 1.0),
)

figure.suptitle("Runtime comparison over all dimensions", y=1.03)

figure.tight_layout()
figure.savefig(
```

```
    FIGURES_DIR / "runtime_comparison.png",
    dpi=300,
    bbox_inches="tight",
    metadata={"Software": None},
)
plt.close(figure)

print("wrote runtime_comparison.png")
```

5.2 Results So Far

To date, benchmarks have been run for a single fixed dimension, $D = 10$, over input sizes ranging from $N = 100$ to $N = 100{,}000$ points. The raw measurements are reproduced in Table 1.

Table 1: Execution time of MAXIMA2 for $D = 10$ from an earlier measurement run; the table is retained as a record and is regenerated by the current benchmark driver.

N (points)	Execution time (μs)
100	11,692.4
500	70,702.9
1,000	193,519
5,000	1,873,410
10,000	4,995,070
50,000	37,591,500
100,000	79,678,700

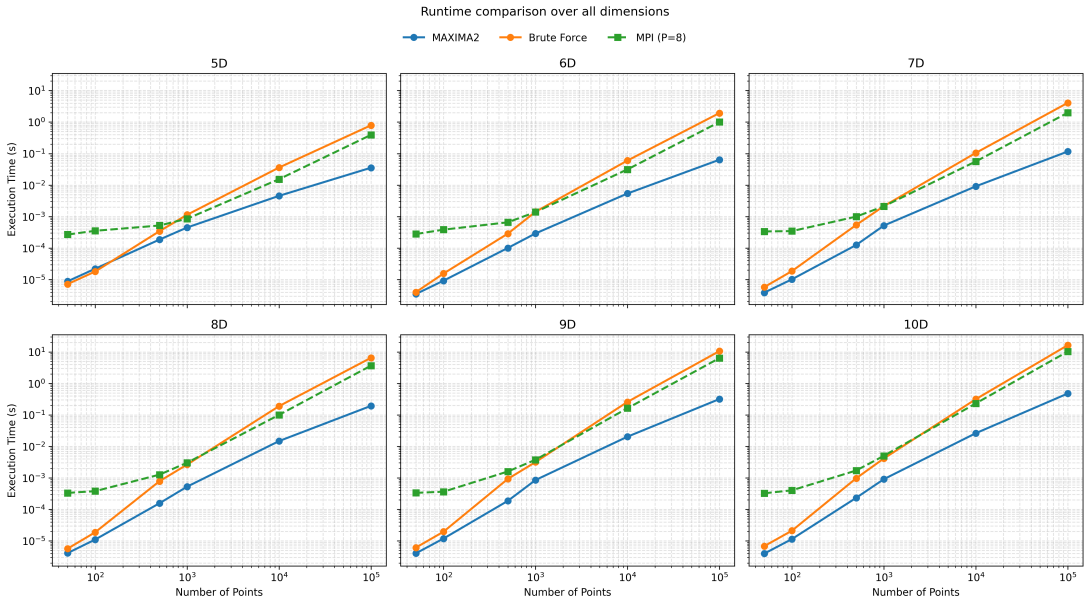


Figure 1: Execution time of MAXIMA2 versus number of points, for $D = 10$ (log-log scale).

5.3 Benchmarking Across Dimensions

Each driver takes a dimension folder and covers the full series of input sizes for that dimension, so runtime can be compared across both N and d. Generated inputs are available for five through ten dimensions; smaller dimensions can be produced with the generator script if needed.

6 Progress Summary

- **Completed:** dominance relation, random data generation, MAXIMA2 divide-and-conquer driver (with brute-force fallback), full recursive FILTER/FILTER2 merge step, an in-place range-based implementation of both algorithms, a validating point-set interface, CMake build system with the benchmark drivers, two test programs and three MPI programs, correctness validation against 16 hand-crafted test cases plus a randomized differential test, and a benchmarking pipeline with automated plot generation.
- **In progress / next steps:** run the benchmark suite over the full range of dimensions and compare the measured scaling with the theoretical bound.

7 Future Scope

The following extensions are planned for the next phase of the project:

- **MPI (OpenMPI) implementation** — three distributed strategies (ring, fan-in/fan-out, marry-go-round) have been implemented for the pairwise formulation. Distributing the divide-and-conquer recursion of MAXIMA2/FILTER itself remains open, with particular attention to load-balanced merge steps.
- **CUDA implementation** — parallelizing the dominance checks and the sweep/merge steps on the GPU for large point sets.
- **Performance evaluation** — measuring speedup and scale-up of the parallel (MPI/CUDA) versions relative to the sequential baseline, across varying number of points N, dimension d, and number of processes/threads/GPU cores, and comparing strong versus weak scaling behavior for both parallelization strategies.